Circuit Switchers: Direct-Mapped Cache Controller

Modern Computer Architecture - Project 6

Guru Manohar Gupta, Harsith Priyan, Pranavi Mathur
Yatin Singh, Dadi Dinesh
Department of CS and AI
Newton School of Technology
Phase I – 12 November 2026

***Abstract*—Processor performance can be limited by the time required to access main memory. A cache retains recently used blocks near the processor and can exploit temporal and spatial locality. Circuit Switchers proposes a direct-mapped, read-only cache controller in Verilog with 32-bit byte addresses, 1024 bytes of data capacity, 64 lines and 16-byte blocks. Direct mapping provides a simple indexed lookup and a deterministic replacement location. The Phase-I prototype implements the address decoder and the data, tag and valid-bit storage; a top-level skeleton establishes the remaining interfaces. Re-executed Icarus Verilog simulations passed six directed decoder cases and 22 storage checks. Stub-assisted elaboration verifies module-port compatibility only. Functional lookup, byte selection, controller sequencing, main-memory service and integrated-system verification remain pending. This report presents the implemented architecture, the proposed transaction flow, verified component evidence and the five-member work allocation without claiming completed cache operation or measured performance gains.**

***Index Terms*—direct-mapped cache, Verilog, memory hierarchy, RTL verification, Phase-I prototype**

I. Introduction

Memory hierarchy design balances access latency, capacity and cost [1], [2]. Caches exploit temporal reuse and nearby accesses. A hit serves a request from an existing line; a miss requires a lower memory level. Benefits depend on workload and implementation timing and must be measured.

Circuit Switchers is a five-member Modern Computer Architecture project. Its initial scope is one outstanding read, an 8-bit response, synchronous active-high reset and aligned 128-bit refill. Writes, coherence and physical DRAM integration are outside this milestone. This report audits baseline commit `370a8f7`, with component tests rerun on 12 November 2026. Documentation changes do not alter the RTL. The submission deadline is 16 November 2026.

II. Literature Review

A. Memory hierarchy and locality

Smith's survey examines cache placement, replacement, line size, update policies and trace-driven evaluation [1]. Temporal locality motivates retaining recently accessed data; spatial locality motivates transferring a block containing adjacent bytes. These concepts explain the intended repeated-address and sequential-address demonstrations. Historical performance figures in the survey are not measurements of this project.

Hennessy and Patterson place memory hierarchy design within quantitative architectural evaluation [2]. Cache capacity, access time and miss behavior must be assessed together. The present project adopts a small fixed geometry to make address fields and replacement effects easy to inspect. It does not claim that 1 KB or a 16-byte block is optimal for a particular application.

B. Direct mapping and hit/miss behavior

Direct mapping assigns each memory block to exactly one cache line [1], [2]. An index selects that line, while a stored tag distinguishes blocks sharing the index. A valid bit indicates whether the stored contents are usable. This organization simplifies placement and avoids selecting among multiple ways, but competing addresses can repeatedly evict one another.

The proposed design uses validity and tag equality to identify a hit. On a miss, its intended controller waits for a memory block and replaces the indexed line. The literature establishes the cache principles; the specific signal names, byte-lane convention and partial implementation documented here come from the team's repository.

C. Hardware description and verification

IEEE Std 1364-2005 defines Verilog as a hardware description language supporting design and verification [3]. Published on 7 April 2006, it is cited as the historical Verilog specification, not as the current SystemVerilog standard.

The prototype separates combinational decoding and reads from rising-edge storage writes. Self-checking testbenches compare observed values with expected values and terminate on failure. The existing scripts use Icarus Verilog's `-g2012` mode, which accommodates the testbench failure task; this audit does not claim strict Verilog-2005 tool conformance. Compiling a hierarchy with constant-output stubs cannot demonstrate functional memory transactions.

III. Proposed Solution

A. Configuration and address fields

The implemented geometry is summarized in Table I. The data capacity is 64 x 16 = 1024 bytes. Six index bits select 64 lines, four offset bits select 16 bytes and the remaining 22 bits form the tag. Metadata adds 64 x (22 + 1) = 1472 bits, or 184 bytes; the advertised 1 KB refers to data capacity alone.

Fig. 1 gives an architectural overview with implementation status. Fig. 2 shows the exact assignments in `rtl/address_`

decoder.v: tag = address[31:10], index = address[9:4], and offset = address[3:0]. For example, 0x00000000 and 0x00000400 both select line 0, with tags 0 and 1 respectively. The decoder test verifies this mapping; it does not demonstrate an actual conflict miss.

TABLE I
IMPLEMENTED CACHE CONFIGURATION

Property	Value
Address	32-bit byte address
Data capacity	1024 bytes (1 KB)
Lines / block	64 / 16 bytes (128 bits)
Tag / index / offset	22 / 6 / 4 bits
Data / tag / valid storage	8192 / 1408 / 64 bits
Intended processor response	8-bit read; interface only

B. Implemented storage and reset

cache_data_ram stores 64 128-bit blocks and tag_ram stores 64 22-bit tags. Both expose independent read and write indexes, combinational reads, and synchronous writes gated by write_en. They have no reset ports; before initialization, their values may be unknown in simulation.

valid_bit_ram stores one bit per line. Active-high synchronous reset clears all 64 bits and takes priority over a write. An enabled write can set or clear one bit. In cache_top, the intended refill path ties write_valid to one. Correct hit logic must reject an invalid line regardless of its tag or data; that hit logic is still absent.

C. Existing top-level connections

cache_top contains real module instantiations and interconnecting wires, but only the decoder and three storage implementations are present. It wires req_addr into the decoder, decoded fields into cache_controller, and controller-held active_* fields into the storage and lookup paths. Fig. 5 records the signal-level connections; repeated controller boxes refer to the same instance.

Two data_selector instances are wired: one selects from stored_block, and the other from mem_block. Each is intended to select byte lane n as block_data[(8*n) +: 8], so offset zero selects bits [7:0]. Neither selector has functional RTL. The assignment mem_addr = {active_tag, active_index, 4'b0000} exists and aligns a future memory request to 16 bytes.

Missing top-level module types are cache_lookup, data_selector, cache_controller and main_memory_model. The contract additionally assigns a tag_comparator inside the future lookup implementation; it is not an existing top-level instance. Files in tb/stubs/ provide constant outputs only and are excluded from claims of cache functionality.

D. Intended hit, miss and refill operation

Fig. 3 is the proposed transaction flow. When req_valid and req_ready are high at a rising edge, the controller must capture tag, index and offset and hold them until completion. The selected storage line can then be checked. The intended hit equation is stored_valid AND (active_tag = stored_tag). A hit should return the selected cached byte with resp_hit high and without a memory request.

A miss should assert mem_req and hold the aligned address until mem_ready. While the returned block is valid, one refill edge must update data, tag and valid storage at active_index. The response byte should be selected directly from mem_block, with resp_hit low. resp_valid must pulse exactly once for each accepted request. All of this sequencing remains planned; unit-tested storage writes are not evidence of completed refills.

Fig. 4 is explicitly a proposed state model. It uses the illustrative names IDLE, LOOKUP and MEM_WAIT already mentioned in the controller requirements. The transitions express required phases, not extracted state encodings or measured cycle counts. In particular, adding separate REFILL or RESPOND cycles requires retaining the returned block or revising the timing contract; the current memory interface promises valid data only with its ready pulse.

IV. VERIFICATION AND PRELIMINARY RESULTS

A. Executed component checks

Table II summarizes the rerun results. The decoder cases cover zero, an interior byte offset, the last byte of a block, the next block, a different tag at the same index and the maximum 32-bit address. Storage checks cover reset of all valid bits, reset/write priority, data and tag readback and replacement, line isolation, boundary data lines, disabled writes, explicit valid-bit clearing and reset after population.

These are directed checks, not exhaustive coverage. Storage data patterns exercise lines 0, 3, 4 and 63; tag checks focus on lines 3 and 4. Valid-bit loops inspect all 64 entries for specified reset/isolation conditions. The two testbenches are independent and do not issue processor-level cache transactions.

TABLE II
OBSERVED AUDIT RESULTS

Check	Observed result
Decoder unit simulation	6 run; 6 passed; 0 failed
Storage unit simulation	22 run; 22 passed; 0 failed
Top level with stubs	Elaboration succeeded
Top level without stubs	Exit 6; four module types absent
Full-cache transactions	Not run; dependencies absent

B. Reproducibility and limits

The tool versions, source hashes, commands and exit statuses are recorded in results/phase1_evidence.json. The two selected final VCDs and component logs are retained in Git. Running python3scripts/capture_phase1_evidence.py recreates this component evidence. The missing-module log documents why a functional top-level simulation is blocked.

Icarus reports timescale warnings for the decoder and top-level skeleton. These do not invalidate the observed directed checks, but simulation time units should be made explicit before integrated timing analysis. The storage testbench clock period is a stimulus choice, not a demonstrated hardware clock frequency.

No integrated hit count, miss count, random workload, AMAT, synthesis utilization or maximum clock frequency has been measured. For future reporting, define miss rate as misses/accepted requests and use AMAT = hit time + miss rate x additional miss penalty [2], with consistent units and a stated sampling method. The additional penalty must exclude the hit-time term to avoid counting it twice.

V. Individual Contributions

The team supplied the names and institution. Assigned responsibility is distinguished from delivered work. Git history supports file changes by Members 01 and 02; no implementation or system-test delivery was found for Members 03-05. Audit documentation and evidence capture are not retroactively credited as their implementation work.

A. Member 01 - Guru Manohar Gupta

Assigned: address decoder, top-level integration, interface contract and architecture documentation. Delivered: `rtl/address_decoder.v`, `rtl/cache_top.v`, the decoder testbench, interface/architecture documents and compile scripts. Commits `9aa6577`, `d930302`, `1217060` and `370a8f7` appear under Git author GuruManoharGuptaBaratam. Six decoder checks pass; the skeleton elaborates with stubs. The contribution establishes address interpretation and module boundaries. Full integration remains incomplete.

B. Member 02 - Harsith Priyan

Assigned: Cache Data RAM, Tag RAM and Valid-Bit RAM. Delivered: `rtl/cache_data_ram.v`, `rtl/tag_ram.v`, `rtl/valid_bit_ram.v`, `tb/tb_cache_storage.v`, storage documentation and a passing log. Commits `5662211`, `f434789`, `114d788` and `d6e5ccc` appear under Git author Harsith Priyan Sivanandham. All 22 directed storage checks pass. These modules provide the cache's data and metadata storage; integrated refill behavior is not yet verified.

C. Member 03 - Pranavi Mathur

Assigned: Tag Comparator, Cache Lookup and Data Selector. Planned files are `rtl/tag_comparator.v`, `rtl/cache_lookup.v` and `rtl/data_selector.v`; none exists in the audited RTL directory. No functional verification for this assignment is available. The future contribution will validate tag identity, gate hits with the valid bit and select an 8-bit byte from a 128-bit block. Constant-output stubs are interface placeholders only.

D. Member 04 - Yatin Singh

Assigned: Cache Controller FSM, Main-Memory Model, memory request and refill sequencing. Planned files `rtl/cache_controller.v` and `rtl/main_memory_model.v` are absent. Requirements exist in `docs/controller_requirements.md` and the shared contract, but they are not delivered controller RTL. No state-transition, memory-response or refill test has run. This work must implement request capture, stable handshakes, refill timing and exactly-once completion.

E. Member 05 - Dadi Dinesh

Assigned: system testbench; sequential, repeated, seeded random and conflict workloads; hit/miss reporting; AMAT; synthesis and performance evidence. `docs/system_test_requirements.md` defines the hand-off, but no full-system testbench or performance/synthesis results are present. Existing decoder and storage tests belong to the delivered Member 01/02 work. The future contribution is to demonstrate data correctness and quantify complete-cache behavior after integration.

VI. Conclusion and Next Milestone

Phase I provides a verified decoder, tested cache storage and an explicit integration contract. The geometry matches the intended direct-mapped cache. Five Mermaid sources separate actual boundaries from pending behavior.

The next milestone is lookup/selection delivery, controller/memory integration and system regression. Tests must cover cold misses, repeated hits, sequential bytes, conflicts, all byte lanes, returned-data correctness, handshakes and reset before performance results are published.

The video remains unrecorded. Before submission, the team must record a demonstration of at most five minutes, introduce every member by name, show current prototype progress and replace the video-link placeholder. The readiness checklist accompanies this report; no final spoken script is included.

References

[1] A. J. Smith, "Cache Memories," ACM Computing Surveys, vol. 14, no. 3, pp. 473-530, Sep. 1982. [Online]. Available: https://courses.grainger.illinois.edu/ece511/fa2005/papers/Smith.1982.CSUR.pdf

[2] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Morgan Kaufmann, 2017. [Online]. Available: https://shop.elsevier.com/books/computer-architecture/hennessy/978-0-12-811905-1

[3] IEEE, IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005, Apr. 2006, doi: 10.1109/IEEESTD.2006.99495. [Online]. Available: https://ieeexplore.ieee.org/document/1620780

APPENDIX A
MERMAID ARCHITECTURE FIGURES

All diagrams retain editable Mermaid sources. Green marks implemented and tested primitives; amber marks partial wiring; dashed gray marks pending behavior. The controller state model is proposed only.

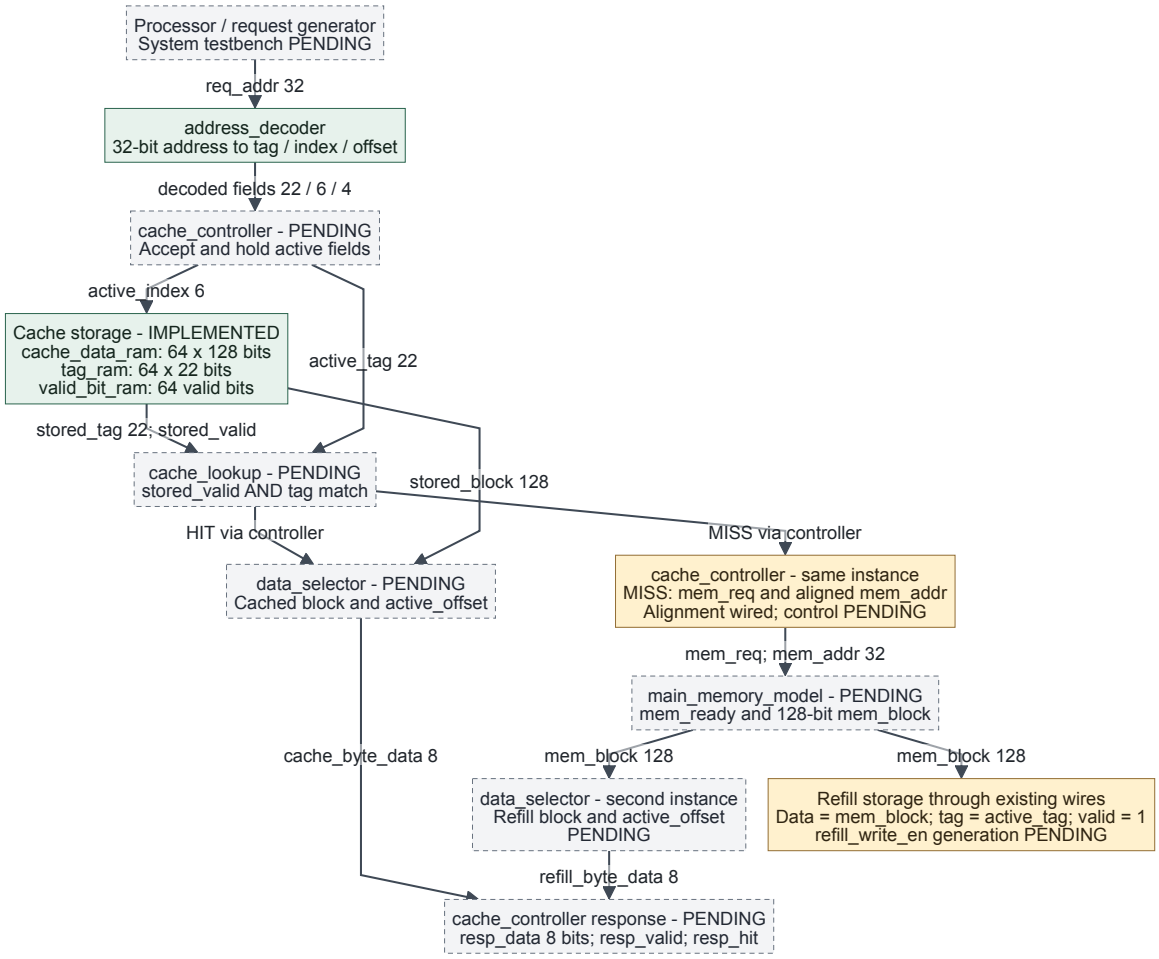


Fig. 1. System architecture based on cache_top boundaries. Green denotes implemented and independently tested primitives; amber denotes existing wiring with pending control; dashed gray denotes pending behavior. Repeated controller/storage views describe the same boundaries, not extra RTL instances.

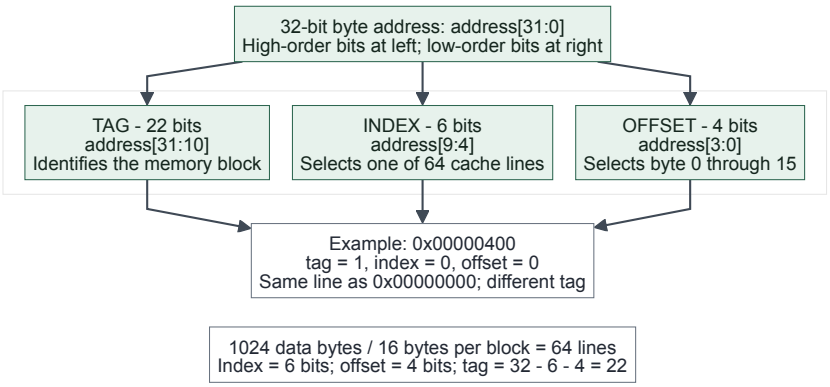


Fig. 2. Implemented 32-bit address split from address_decoder.v. The tag distinguishes blocks sharing an index; the index selects a line; the offset identifies a byte. The worked example is a decoded address, not an observed cache miss.

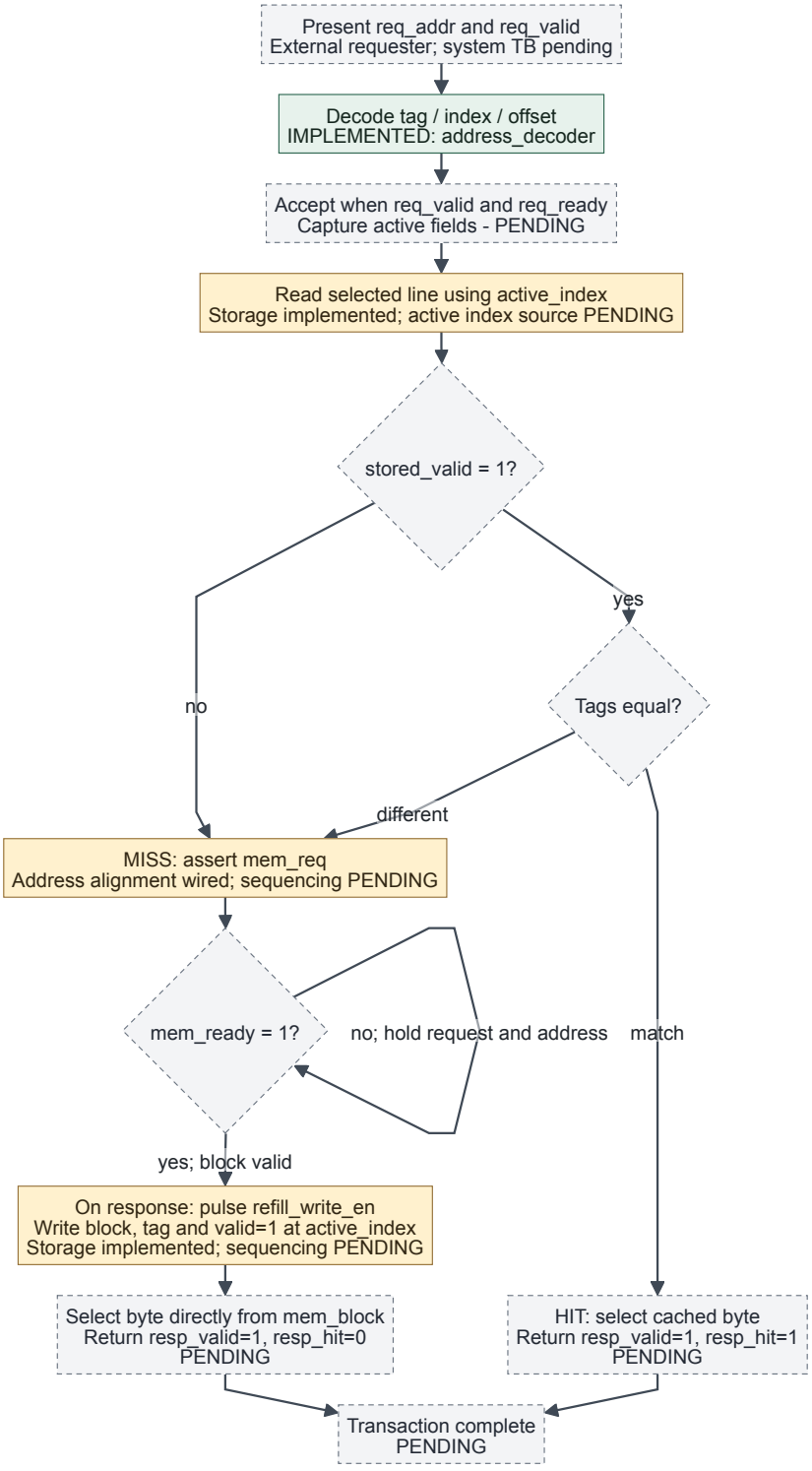


Fig. 3. Proposed cache transaction flow from the interface contract. Address decoding and storage primitives exist. Acceptance, comparisons, hit/miss handling, memory service, refill sequencing and completion remain pending. Dashed decision nodes are not implemented logic.

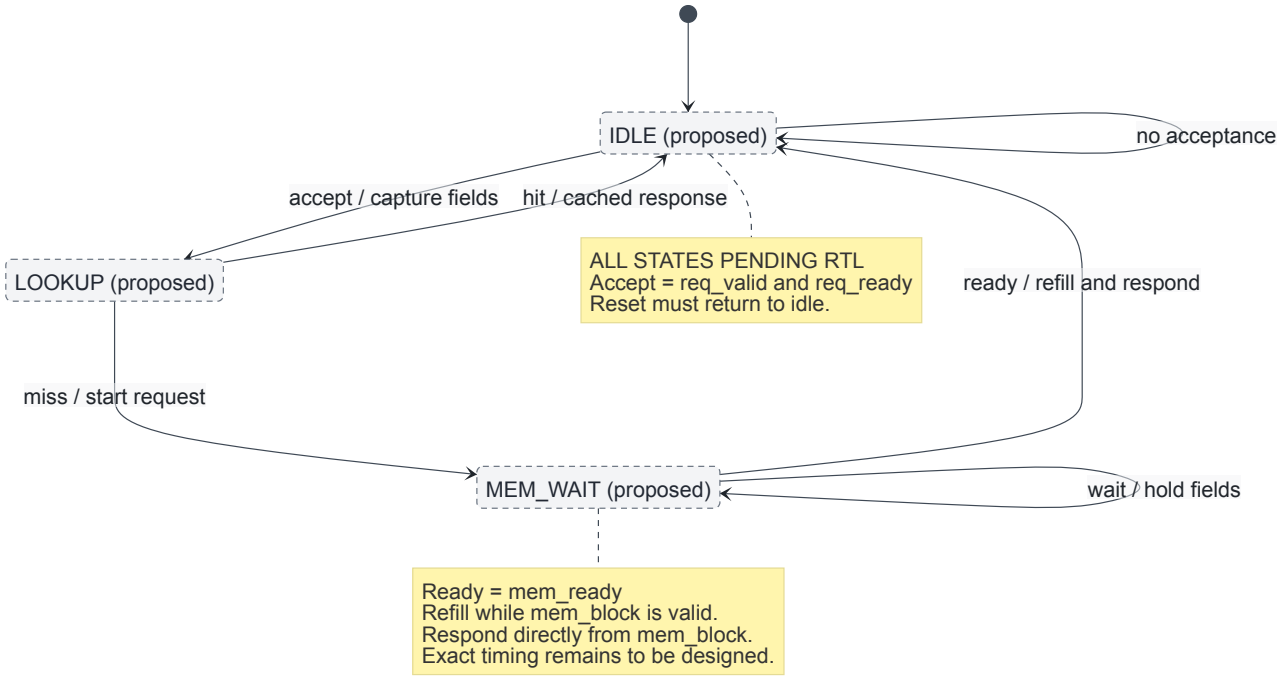
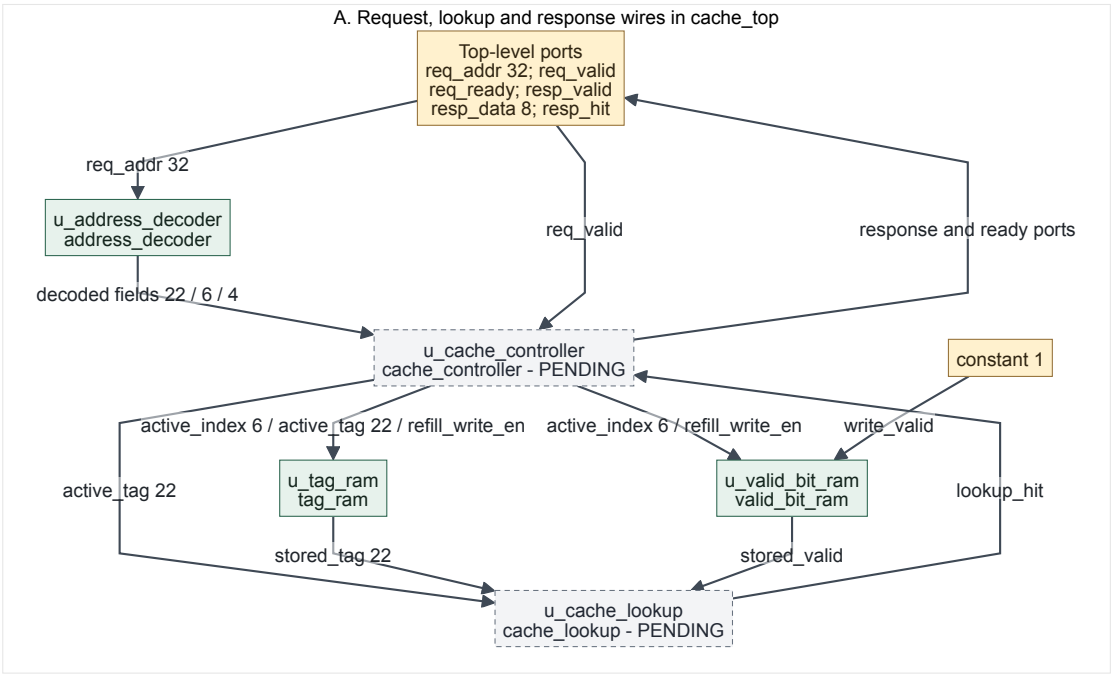
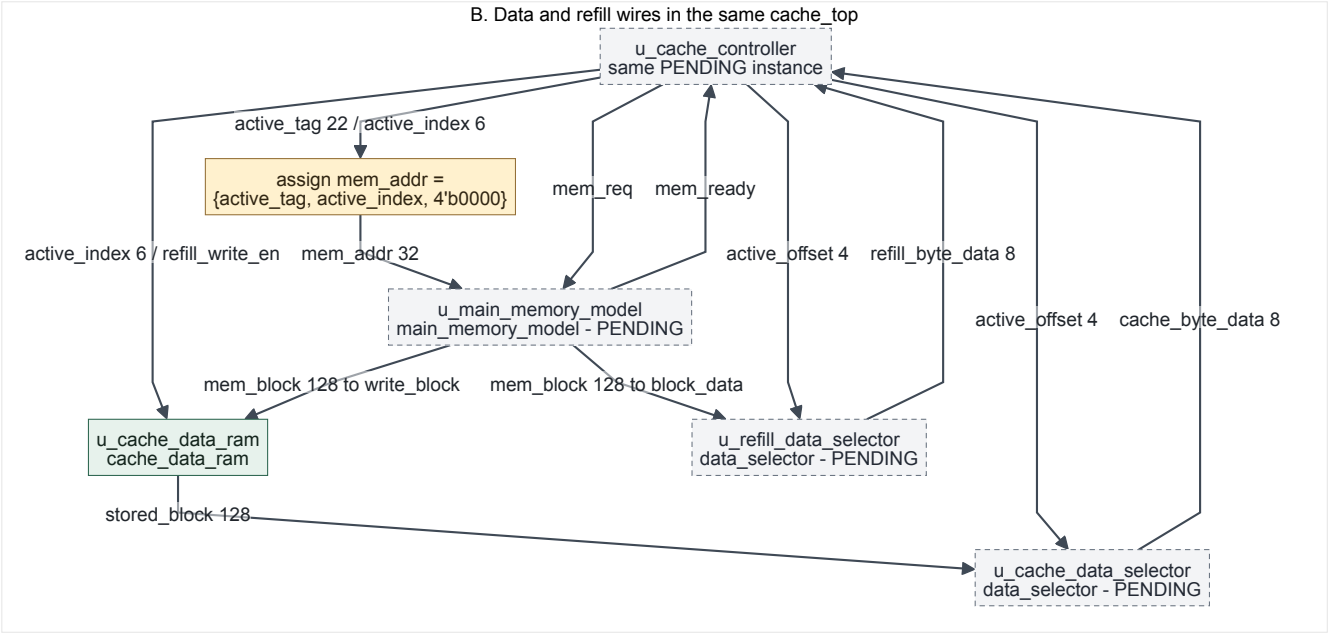


Fig. 4. Proposed controller state model, not extracted RTL. All states and transitions await implementation. The illustrative names originate in controller_requirements.md; the refill/response transition must consume a valid returned block and preserve one response per request.





Clock: all RAMs, controller, memory
Reset: valid-bit RAM, controller, memory
Data / tag RAM have no reset ports.
tag_comparator is planned inside lookup; no existing instance.

Fig. 5. Actual module instantiations and signal relationships in cache_top.v. Views A and B repeat the same controller for readability. Dashed boxes are missing functional dependencies. The comparator is planned inside lookup and has no existing instance; common clock/reset connections are listed in the diagram.